

Video Insight Studio

Project Report & Interview Preparation Guide

An AI-powered video/audio summarizer with Retrieval-Augmented Generation (RAG) chat, built with FastAPI, LangChain, Whisper, Chroma, and React.

Generated August 06, 2026

1. Executive Summary

Video Insight Studio lets a user upload a video/audio file or paste a public video URL and receive an AI-generated title, bullet-point summary, key points, action items, and key moments. Under the hood, speech is transcribed locally with OpenAI's Whisper model, and the transcript is embedded into a Chroma vector store so the user can then hold a grounded, Retrieval-Augmented Generation (RAG) conversation with the video — asking follow-up questions that get answered from the transcript rather than the model's general knowledge. The full report and transcript can be exported as a PDF, and the original media file can be downloaded directly from the results page.

Resume bullet points (ready to use):

- Built a full-stack AI video/audio summarizer (FastAPI + React) combining local speech-to-text (Whisper), LLM summarization (LangChain + Mistral/Ollama), and a Retrieval-Augmented Generation chat feature grounded in a Chroma vector store.
- Designed a swappable LLM provider layer that prefers a free local Ollama model and falls back to the Mistral cloud API, keeping the app free to run locally and cheap to deploy.
- Diagnosed and fixed a production-breaking bug where every RAG query silently failed due to a duplicate keyword argument in the embeddings client, using end-to-end testing against a live server to catch it.
- Found and fixed a data-isolation bug where the vector store accumulated chunks across multiple analyzed videos, causing RAG answers to leak context from unrelated videos.
- Containerized the backend with Docker to guarantee ffmpeg availability on Render (missing from the native Python runtime), and pre-baked model weights into the image to remove cold-start download latency.

2. System Architecture

Request flow:

Frontend (React/Vite) -> FastAPI backend -> [yt-dlp / ffmpeg] -> Whisper (speech-to-text) -> LangChain + LLM (title/summary/key points/action items) -> Chroma (embeddings) -> RAG chat / PDF export / media download.

Backend module map:

```
main.py - FastAPI routes: /upload, /process, /chat, /download/transcript, /download/media, /health
utils/audio_processor.py - yt-dlp download, ffmpeg chunking, caption scraping
config/transcriber.py - Whisper speech-to-text
config/translator.py - title + summary generation (map-reduce over chunks)
config/extractor.py - key points / action items / key moments extraction
config/vector_store.py - Chroma embeddings + retriever
config/ragEngine.py - RAG chat chain (retrieve -> prompt -> LLM)
config/llm.py - Ollama (local) / Mistral (cloud) provider switch
config/pdf_export.py - reportlab PDF generation for transcript export
```

Tech stack:

Layer	Choice	Why
Backend API	FastAPI + Uvicorn	Async, typed, auto-generated OpenAPI docs
Speech-to-text	OpenAI Whisper (local)	No per-minute STT API cost; runs offline
Orchestration	LangChain (LCEL)	Composable prompt llm parser chains
LLM provider	Ollama (local) / Mistral (cloud)	Free locally, reliable fallback in production
Vector store	Chroma (local, persisted)	Zero-ops embedded vector DB, good for a single-tenant demo
Embeddings	sentence-transformers MiniLM-L6-v2	Small, fast, strong quality/size trade-off
Frontend	React 19 + Vite	Fast dev server, small bundle, modern hooks-only code
PDF export	reportlab	Pure-Python, no external binary dependency
Deployment	Render (Docker backend + static frontend)	Docker guarantees ffmpeg is present

3. How It Works (Step by Step)

1. Ingest

User uploads a file (multipart POST /upload) or pastes a URL (POST /process). Uploaded files are validated by extension and streamed to disk under uploads/.

2. Get a transcript

For URLs, yt-dlp first tries to pull existing captions (fast path, no audio download needed). If that fails, audio is downloaded/extracted, converted to mono 16kHz WAV, and split into 5-minute chunks via a direct ffmpeg subprocess call.

3. Transcribe

Each chunk is run through a locally loaded Whisper model (config/transcriber.py) and the text is concatenated into a full transcript.

4. Analyze

The transcript is sent through four independent LangChain chains (title, summary, key points, action items/key moments). The summary chain uses a map-reduce pattern: it splits the transcript, summarizes each chunk, then combines those chunk summaries into one final summary — keeping token usage bounded on long transcripts.

5. Embed

The transcript is chunked (500 chars, 50 overlap) and embedded with a sentence-transformers model into a Chroma collection persisted to disk. The previous collection is deleted first so a new video's chunks don't mix with the last one's.

6. Chat (RAG)

POST /chat embeds the user's question implicitly via the retriever, pulls the top-k most similar transcript chunks, and feeds them as context into a prompt so the LLM answers strictly from the video's content instead of general knowledge.

7. Export

GET /download/transcript renders the session's title/summary/points/transcript into a formatted PDF on the fly. GET /download/media streams back the original file that was analyzed (the uploaded file, or the downloaded YouTube audio when no captions existed).

4. Key Engineering Decisions

Why a local Whisper model instead of a cloud STT API?

Keeps the app free to run and demo without an STT vendor key. The trade-off is CPU/RAM cost at inference time, which is why the model size is configurable via `WHISPER_MODEL` (tiny/base/small) instead of hardcoded.

Why does `config/llm.py` support two providers (Ollama and Mistral)?

Local development can run entirely offline/free against Ollama. Production (Render) has no GPU/local model server, so it needs to fall back to a hosted API automatically. The probe checks for a running Ollama server and installed models first, then falls back to Mistral — with a `DISABLE_OLLAMA` flag to skip that network probe entirely in the cloud.

Why is the vector store's collection reset on every new upload?

`Chroma.from_documents()` appends to an existing collection rather than replacing it. Without resetting first, chunks from every video ever analyzed in that process would stay queryable, and RAG chat would silently mix answers from unrelated videos — a real bug found during testing (see Section 5).

Why move the backend to a Dockerfile instead of Render's native Python runtime?

The app depends on the `ffmpeg` binary (via `pydub` and a direct subprocess call for chunking), which Render's native Python buildpack doesn't guarantee. A Dockerfile installs `ffmpeg` explicitly via `apt-get`, so behavior is reproducible between local dev and production.

Why bake Whisper/embedding model weights into the Docker image at build time?

Without this, the first request after every deploy pays a multi-minute weight-download penalty (and fails outright if the download is interrupted). Baking them in trades a larger image / longer build for a fast, reliable first request.

5. Bugs Found & Fixed During Testing

These were found by actually running the app end-to-end (upload -> transcribe -> summarize -> RAG chat -> PDF/media download) rather than just reading the code — good material for "tell me about a bug you found" interview questions.

RAG was completely broken in production

`get_embedding()` passed `cache_folder` inside `model_kwargs` to `HuggingFaceEmbeddings`. The wrapper already forwards its own `cache_folder` parameter to the underlying `SentenceTransformer(...)` call, so the duplicate raised: "SentenceTransformer() got multiple values for keyword argument 'cache_folder'" on every single upload — meaning the vector store, and therefore RAG chat, never worked. Found by running a real upload against the live server. Fix: pass `cache_folder` as its own top-level argument.

RAG answers could leak context from a previous video

`build_vector_store()` always wrote to the same Chroma collection name via `Chroma.from_documents()`, which adds documents rather than replacing them. Analyzing a second video would silently mix its chunks with the first video's in the same collection. Fix: delete the existing collection before building the new one.

CORS misconfiguration

`allow_origins=["*"]` combined with `allow_credentials=True` is rejected by browsers per the CORS spec (a wildcard origin cannot be paired with credentialed requests). The app doesn't use cookies, so the fix was `allow_credentials=False`.

Dead, colliding code

A root-level `app.py` (a standalone Groq-based CLI chatbot, unrelated to the rest of the stack) shared its name with the `app/` package used for `app.main:app` deploy compatibility. Python's import system resolves `import app` to the package, not the module, so `app.py` was silently unreachable dead code — removed.

Missing dependency

`config/vector_store.py` imports `HuggingFaceEmbeddings`, which requires the `sentence-transformers` package at runtime, but it was never listed in `requirements.txt` — a guaranteed `ImportError` on a clean deploy. Added it (along with `reportlab` for PDF export).

Unused heavy dependencies bloating the build

`moviepy`, `opencv-python`, `nltk`, and `pandas` were listed in `requirements.txt` but never imported anywhere in the codebase, needlessly inflating install time and Docker image size. Removed.

Messy LLM-generated titles

The title-generation prompt didn't forbid markdown, so titles sometimes came back as **"Quarterly Budget Review"** — asterisks and quotes leaking into the UI heading and into the downloaded PDF's filename. Fixed with an explicit "plain text, no markdown/quotes" instruction plus a defensive `strip()` as a safety net.

6. Deployment Guide (Render)

render.yaml defines two services: a Docker-based FastAPI backend (video-summarizer-backend) and a static React frontend (video-summarizer-frontend). Key steps:

- Push to GitHub, create a Render Blueprint pointing at the repo.
- Set MISTRAL_API_KEY (required) and optionally HF_TOKEN as backend secrets in the Render dashboard — they're marked sync: false so Render prompts for them instead of committing them.
- After the backend's first deploy, copy its onrender.com URL and set it as VITE_API_URL on the frontend service, then redeploy the frontend (Vite bakes VITE_* vars in at build time, not runtime, so this step can't be skipped).
- Resource notes: Whisper + PyTorch + sentence-transformers + Chroma is a heavy stack for a free-tier instance. Keep WHISPER_MODEL=tiny/base, and upgrade to a paid instance type if you see build timeouts or OOM restarts.
- Storage is ephemeral on Render without an attached persistent disk: uploads, downloads, and the vector store reset on every deploy/restart. Expected for a single-session demo.

7. Interview Questions & Answers

Organized by topic. Answers are written specifically for this project's actual implementation, not generic definitions — use them as a base and speak to the real code.

7.1 — Project Walkthrough & Behavioral

Walk me through what happens when a user uploads a video.

The file streams to disk via FastAPI's `UploadFile`, gets validated by extension, then `analyze_source()` runs it through `ffmpeg` chunking, Whisper transcription, four parallel `LangChain` extraction chains (title/summary/key points/action items), and finally embeds the transcript into Chroma. The full result is returned to the frontend as JSON, and the session state (transcript, media path) is cached server-side for later `/chat` and download calls.

What was the hardest bug to track down, and how did you find it?

The RAG pipeline silently failing — every summary/title endpoint worked because they don't touch the vector store, so the bug only surfaced on `/chat` or in server logs during `build_vector_store()`. I found it by actually exercising the full flow end-to-end with a real audio file and reading the raised exception (`'SentenceTransformer() got multiple values for keyword argument cache_folder'`), which pointed straight at the duplicate `kwargs`.

What would you do differently if you rebuilt this?

Add an integration test that uploads a short fixture audio file and asserts `/chat` returns a grounded answer — that single test would have caught both the `cache_folder` bug and the vector-store-bleed bug immediately instead of relying on manual end-to-end testing.

Why did you choose this project to showcase?

It touches a full slice of a modern AI product: audio/ML pipeline (Whisper), LLM orchestration (`LangChain`), retrieval systems (embeddings + vector DB + RAG), a REST API (FastAPI), a React frontend, and real deployment constraints (Docker, env vars, resource limits) — enough surface area to talk concretely about trade-offs in each layer.

7.2 — Python & FastAPI

Why FastAPI over Flask/Django here?

Native async support (useful for I/O-bound calls to the LLM API and file streaming), Pydantic request validation out of the box (`VideoRequest/ChatRequest` models), and automatic OpenAPI docs at `/docs` with almost no boilerplate.

How does file upload validation work, and why check the extension instead of content-type?

`validate_extension()` checks the file's suffix against an explicit allow-list (`ALLOWED_EXTENSIONS`). Browsers/clients often send unreliable or missing `Content-Type` headers for multipart uploads, whereas the filename extension is simple and sufficient here since the real gatekeeper is `ffmpeg` failing loudly on an unsupported file anyway.

Where does global mutable state live, and why is that acceptable here?

A module-level `SESSION` dict in `main.py` holds the most recently analyzed video's transcript, summary fields, and media path. It's acceptable because this is explicitly a single-tenant demo app — documented as a known limitation — but it would need to become per-request/per-user state (e.g. a

session ID + database or cache) to support concurrent users safely.

How are errors surfaced to the client?

Route handlers catch exceptions and re-raise as `HTTPException` with a status code and a detail string that the frontend reads via `err.response.data.detail` and shows inline.

7.3 — LangChain, RAG & Vector Databases

Explain the RAG flow in this app end to end.

At ingest time, the transcript is chunked (500 chars, 50 overlap) and each chunk is embedded with a sentence-transformers model, then stored in a Chroma collection. At query time, the user's question is embedded the same way, Chroma returns the top-k (k=4) most similar chunks by cosine similarity, and those chunks are concatenated as 'Context' in a prompt template alongside the question, then sent to the LLM — so the model answers from retrieved transcript text instead of parametric memory.

Why chunk the transcript instead of embedding the whole thing?

Embedding models have a limited effective context window and produce better, more specific similarity matches over smaller passages. Chunking with overlap (50 chars here) also avoids losing meaning at chunk boundaries.

Why Chroma instead of Pinecone/Weaviate/pgvector?

Chroma runs embedded, persisted to local disk, with zero external infrastructure to provision — appropriate for a single-tenant demo. A production multi-tenant version would likely move to a managed vector DB (or pgvector alongside existing relational data) for durability, horizontal scaling, and metadata filtering per user/session.

What is LCEL (LangChain Expression Language) and where is it used here?

It's LangChain's pipe (`|`) syntax for composing runnables into a chain. `build_chain()` in `config/extractor.py` composes a prompt template, the LLM, and an output parser as `prompt | llm | StrOutputParser()`; `ragEngine.py` composes the retriever, prompt, and LLM similarly, with retriever output piped through a formatter that joins document text.

How would you evaluate RAG answer quality?

In order of effort: manual spot-checking against known transcript facts (what I did here), then a small labeled Q&A eval set with an LLM-as-judge scoring relevance/faithfulness, then production feedback signals (thumbs up/down) if this became a real product.

What's the map-reduce summarization pattern used for, and why?

`summarize()` in `config/translator.py` splits a long transcript into chunks, summarizes each chunk independently ('map'), then combines those summaries into one final summary ('reduce'). This keeps prompt size bounded regardless of video length, instead of trying to stuff an arbitrarily long transcript into one LLM call.

7.4 — Whisper & Audio Processing

Why chunk audio into 5-minute segments before transcribing?

Bounds memory/latency per Whisper call and lets long files be processed incrementally instead of requiring the whole file to fit in memory/inference time at once.

Why prefer YouTube captions over downloading and transcribing audio?

It's dramatically faster and cheaper (no audio download, no ffmpeg pass, no local model inference) when the video already has accurate captions available — `get_youtube_transcript()` tries that first and only falls back to full audio transcription if no usable captions exist.

What are the trade-offs of Whisper 'base' vs a larger model?

Larger models (small/medium/large) are more accurate, especially on accents, noisy audio, or domain vocabulary, but are slower and use more RAM/CPU per request — the model size is kept configurable via `WHISPER_MODEL` specifically so this can be tuned per deployment.

7.5 — Frontend (React)

How does the frontend know which backend to call in dev vs production?

`API_URL` reads `import.meta.env.VITE_API_URL`, defaulting to `'/api'`. In dev, Vite's proxy config rewrites `/api/*` to the local FastAPI server. In production, `VITE_API_URL` is set at build time to the deployed backend's full origin, since Vite static builds have no runtime proxy.

How is the video preview implemented without a server round-trip?

For an uploaded file, the frontend already holds the browser File object, so it creates a local blob URL with `URL.createObjectURL(file)` for instant `<video>` playback, and revokes it on cleanup to avoid a memory leak — no need to re-fetch the file from the server just to preview it.

How do the PDF/media download buttons work?

They're plain `<a href=... download>` tags pointing at the backend's GET endpoints. The browser's download behavior is actually driven by the server's Content-Disposition: attachment header, not the HTML download attribute (which cross-origin browsers ignore anyway) — and because it's a normal navigation rather than a fetch/XHR call, it isn't subject to CORS restrictions.

7.6 — System Design / Scaling

This app currently supports one active session globally. How would you make it multi-user?

Replace the module-level `SESSION` dict with a session ID (issued per browser session or user), keyed lookups into a real datastore (Redis/Postgres) for transcript/results, and a per-session Chroma collection name (or a metadata filter on a shared collection) so RAG retrieval only ever sees that session's chunks.

How would you handle a much larger volume of video processing?

Move transcription/analysis off the request/response cycle entirely: accept the upload, enqueue a background job (Celery/RQ or a cloud task queue), and let the frontend poll or subscribe (WebSocket/SSE) for status instead of blocking an HTTP request for however long Whisper + several LLM calls take.

Where are the current single points of failure?

The in-memory `SESSION` state is lost on any restart; local disk storage for uploads/vector store isn't durable on ephemeral hosts; and there's a single global vector collection with no isolation between concurrent analyses.

How would you add authentication/rate limiting?

Put the API behind a lightweight auth layer (API key or JWT) checked via a FastAPI dependency, and add per-key rate limiting (e.g. `slowapi`/Redis token bucket) especially around `/upload` and `/chat`, since those are the calls that cost real LLM/compute money.

7.7 — DevOps & Deployment

Why Docker for the backend but a native static build for the frontend?

The backend has a real system dependency (ffmpeg) that needs guaranteed installation — Docker makes that explicit and reproducible. The frontend is just static HTML/JS/CSS after `vite build`, so Render's native static-site hosting (with a CDN) is simpler and cheaper than running a container just to serve files.

What was wrong with the original CORS configuration, concretely?

`allow_origins=["*"]` with `allow_credentials=True` violates the CORS spec — browsers refuse to honor a wildcard Access-Control-Allow-Origin on a credentialed request. Since this app never sends cookies/auth headers cross-origin, the correct fix was `allow_credentials=False`, not narrowing the origin list.

Why pre-download model weights in the Dockerfile instead of at first request?

Downloading ~150MB+ of Whisper/embedding weights on the first real user request adds unpredictable latency (or outright failure under network issues) to that user's experience. Doing it once at build time trades a slower CI build for a fast, deterministic runtime.

How are secrets (API keys) handled between local dev and Render?

Locally via a gitignored `.env` file loaded with `python-dotenv`; on Render via dashboard environment variables marked `sync: false` in `render.yaml`, so the Blueprint declares that the key is required without ever committing its value.

8. Possible Future Improvements

- Per-session isolation (session IDs + per-session vector collections) for real multi-user use.
- Background job queue for processing, with progress streamed to the frontend via WebSocket/SSE.
- Persist uploads/results to object storage (S3-compatible) instead of local disk.
- Automated integration test that exercises upload -> chat -> download against a fixture file.
- Swap Whisper for a hosted STT API (or faster-whisper) to cut memory footprint on small instances.
- Basic auth + per-key rate limiting before exposing this publicly.